

42 트랙 — C/C++ CS 기본 → 취업용 웹 캡스톤(pong)

이 트랙은 무엇인가. 42 성격 그대로, C-C++로 컴퓨터 기본기(프로세스·메모리·시그널·동시성·템플릿·소켓)를 손으로 쌓고, 마지막은 **취업으로 직결되는** 풀스택 웹 프로젝트 **pong-pong**으로 단는다. 코드 **재현**이 아니라 **판단력** — L3(아래)만 백지에서 다시 만들고 나머지는 빠르게 통과한다.

읽는 법. 위에서 아래로 한 프로젝트씩 해쳐나가면 그대로 완주된다. 각 프로젝트 블록은 **자주적**이다 — 그 프로젝트의 노트 순서·재구성 방법·L3·검증·합정·완료·바·커밋들이 블록 안에 다 있다. 위로 되돌아갈 필요 없다. 이 문서 하나로 트랙을 완주하도록 썼다 — 다른 문서를 퍼지 않아도 된다.

딱 한 번만 외우는 공통 어휘 (이후 각 블록은 이 말만 쓴다): - L3 = 동시성·메모리·소유권·시그널·안전·예외·안전·프로토콜 등 **폭발·변경·큰 결정**. **답지** 덮고 백지에서 코드+이유를 재구성하고, 실제 모드까지 말로 방어해야 "끝"이다. - L2 = 구조·흐름·책임 분리. **답지** 펼쳐 설명되면 충분(재구성은 선택). - L1 = 보일러플레이트·문법·설정. 읽고 통과. 다시 치지 않는다. - 핵심 동작 = docs/commits/NNN.md (답지)를 덮고, docs/practice/NNN.md (문제의)의 계약만 보고 직접 구현 → 답지와 diff → 네 결정이 다르면 왜 다른지 따진다. 이 "벽에 부딪히는 순간"이 학습이다. - 각 답지 끝의 ## 기억/설명 Level 에 그 커밋의 L3/L2/L1이 적혀 있다. 들어가기 전에 그것부터 본다. - 커밋 규율 (학습 산출물을 커밋할 때 — 이 트랙 전 레포 공통): 제목은 **영어 명령형** <type> (<scope>): ... (type ∈ feat·fix·docs·test·refactor·chore·build·ci·perf), 본문은 한국어 [근거] 왜 / [변경] 무엇 / [검증] 어떻게 (잡일 merge는 면제), AI 공동저자 **트레일러 없음**, 날짜는 KST-author==committer-근무시간 09:00-21:59·분초·랜덤·충족 금지. dual-form 문서는 답지 docs(commits) / 문제지 docs(practice) · 스코프로 나눠 **바뀐 파일만** add(−A) 금지.

전체 경로:

```
linux-admin(M0) → small-shell → format-printer → signal-message-bus → thread-dining
→ stack-sort → stl-container → irc-relay-server → ★pong-pong(캡스톤)
```

왜 이 구성인가(범위 결정). CS 기본에 필요한 것만 남겼다. ray-scene-tracer 제외(그래픽스 적용 연습이라 새 CS 개념이 적음), game-server-foundations/reliability 제외(시스템서버 심화는 기본 범위 밖), 서버는 irc 하나만(BSD 소켓·네트워크 프로토콜 = CS 핵심), pong이 끝나면 **백엔드** 트랙 또는 **프론트** 트랙으로 분기한다. pong에서 TypeScript를 익히므로 **프론트**의 TS 선행이 자연히 채워진다.

1. linux-admin — 작업 규율 게이트(M0)

무엇/왜. 모든 경로의 #0. Git-Conventional Commits-Markdown 문서 규율 + Born2beroot's Linux 운영 정책(사용자·SSH·sudo·방화벽)을 **선언하고 검증**하는 레포다. 여기서 익힌 커밋·문서 규율이 이후 8개 레포 전부터서 그대로 쓰인다 — 그래서 첫 번째다. 이 트랙에서 얻는 것. "어떻게 커밋하고 어떻게 문서를 남기는가"의 몸매 벤 스칸.

장르·리듬. 장르 ①<작업규율 제화형(재구성 아님)·순차(2B). 답지 13면 중 문제지는 2편뿐 — 비율 코드가 거의 없다. 덮어 채우는 게 아니라, 절차를 읽고 네 환경에서 실제로 한 번 한다.

노트 읽는 순서 (linux-admin/notes/). git-cli → conventional-commits → github-workflow → markdown. 네 편을 읽고 바로 적용 — 이 레포를 도는 동안 네 커밋 자체가 규율 연습이다.

재구성·커밋 활용법. 답지를 소설처럼 읽지 말고, 체크포인트 적용 순서(시스템 구성도 → 사용자 → SSH → sudo → 방화벽)를 따라가며 **정책 문서를 네 손으로 다시 쓴다**. 004(hardening 체크리스트·검증기)에서 **SSH 정책 = 접속 통제 / sudo = 권한 상승**을 분리해 설명할 수 있어야 한다.

L3 — 손으로 백지 재구성할 것. 가법다. 코드가 아니라 **규율**. "SSH 제한·sudo 로깅·방화벽 허용 포트를 왜 이렇게 두나"를 한 번 실제로 적용해 보는 것.

검증. make test (문서 키워드 검증) + scripts/check-commit-msg.sh · check-time-line.sh · validate_admin_lab.py. (검증기는 문서 grep 수 준이라 실제 SSH/sudo 동작까진 안 봄)

합정. "답지 덮고 코드 채우기"를 기대하지 마라(문제지 2편뿐, 장르가 다르다). 실제 VM 구성이 아니라 **정책 선언**이 핵심.

완료 바 → 다음으로. 다음 커밋부터 **규율이 저절로 지켜지면** 제화된 것. 영어 명령형 제목 + [근거]/[변경]/[검증] 본문이 손에 붙었으면 small-shell 로.

커밋 메모. 여기가 커밋 규율의 원천이다 — 상단 <커밋 규율>을 이 레포를 돌려 몸에 붙인다(영어 명령형 제목 + [근거]/[변경]/[검증] 본문, AI 트레일러 없음, KST 근무시간). 이 규칙은 이후 전 레포 공통이라 각 블록에서 반복하지 않는다.

2. small-shell — C 대장(미니 셸)

무엇/왜. C99 단일 바이너리 미니 셸. 파싱→확장→실행→builtin 상태를 한 프로그램에서 다룬다. C 트랙의 대장(가장 깊은 선행) — 여기서 프로세스·fd·파이프·fork/exec를 손에 익히면 결가지들이 풀린다. 이 트랙에서 얻는 것. "누가 fd를 열고 누가 닫나"를 소리 내 말할 수 있는 메모리·프로세스 소유권 감각.

장르·리듬. 장르 ①(코드 재구성, 일부 따라만들기)·순차(2B). 답지 11 = 문제지 11(1:1) — 덮고 **본문을 채우는** 전형적 재구성 레포. docs/notes/ 없음 → 막히면 루트 notes/ 1부로 대체.

노트 읽는 순서 (small-shell/notes/). c-language (처음 보는 스택이면 1부 개념부터 깊게) → gnu-readline. 짝 reference-impl/ 를 한 번 빌드해 손에 익힌다.

재구성·커밋 활용법. 커밋을 세 묶음으로 본다 — ① 기반(002~003: shell.h 공용 자료구조, 환경 저장소), ② 입력 정책(004~006: lexer→parser tree→expansion), ③ 실행(007~009: stdin 루프→builtin→fork(pipe/exec). **004~006과 007~009가 L3다**. 문제지를 덮고 lexer/parser/executor 계약을 직접 세운 뒤 답지와 diff.

L3 — 손으로 백지 재구성할 것. fork-exec와 pipe fd 관리(누가 닫나), quote/expansion 섞인 lexer, builtin은 **부모 프로세스에서 실행**(상대 변경 경계), 파이프라 인종료 상태 보존, heredoc/redirection 결합.

검증. make → make test (smoke.sh가 builtin-quote-expansion-pipeline-redirection-heredoc- syntax error 시나리오 검증).

합정. job control-wildcard-command substitution-subshell-arithmetic은 범위 밖. 범위를 "한 줄 명령 실행 + builtin 상태"로 고정하고 육신내지 마라.

완료 바 → 다음으로. 빈 파일에서 fork/pipe/exec 경로와 builtin 부모 실행 경계를 **백지로 다시 짜고** fd 소유권을 말로 방어되면 다음. 그 다음 4개(결가지)는 small-shell에서 익힌 C를 각 주제로 **흔히** 적용한다.

커밋 메모. dual-form 스크프: 답지 docs(commits) / 문제지 docs(practice) 분리 커밋 변경 파일만 add(git -C small-shell add docs/commits docs/practice), 전체 −A 금지.

3. format-printer — C 결가지 ①: 가변인자·포매팅(ft_printf)

무엇/왜. 42 ft_printf. 포맷 문자열을 파싱해 변환 지정자·플래그를 처리하고 정적 라이브러리 libftprintf.a 로 제출. va_list 와 플래그 우선순위는 **짧고 깊은** 주제 하나를 판다. 이 트랙에서 얻는 것. 가변인자 안전 전달 + width/precision/flag의 우선순위 추론.

장르·리듬. 장르 ④ 따라만들기(경향(scaffold-first) + ①·순차(2B). 답지 14 / 문제지 13.

노트 읽는 순서 (format-printer/notes/). ar (정적 라이브러리 아카이브) → c-test-harness. 각 노트에 짝 reference-impl이 있다.

재구성·커밋 활용법. 000(공개 API·빌드 규칙 골격, L1) → 001(counted-write 컨텍스트 t_printf, L2: 모든 변환이 같은 오류 반환) → **002(포맷 필드 파서, L3) → 003~005(변환 9종 dispatch) → 006~008(width 정렬·precision/0-padding·부호/alternate, L3: precision-vs-zero-flag 우선순위) → 009~011(libc 비교·오류 처리)**. 002와 006~008을 덮고 직접 짜다.

L3 — 손으로 백지 재구성할 것. % 뒤 flags→width→precision→specifier 순서 소비, 변환 9종과 flag 조합의 우선순위(width→precision 상호작용, 부호+0패딩 동시), va_list 전달, counted-write sticky error.

검증. make → make test (snprintf 와 바이트·반환값 비교).

합정. length modifier-locale-stdout 외 출력·첫 커밋 체크리스트 밖 변환은 범위 밖.

완료 바 → 다음으로. width/precision/flag 우선순위를 백지에서 표로 재현하고 sticky error를 설명되면 다음.

4. signal-message-bus — C 결가지 ②: 시그널·IPC(minitalk)

무엇/왜. 42 minitalk. SIGUSR1 / SIGUSR2 두 시그널만으로 임의 문자열을 비트 스트림으로 신뢰성 있게 전달하고, ACK/NACK-timeout으로 강화. 시그널 안전과 race-free 대기를 판다. 이 트랙에서 얻는 것. signal-safe 코드와 비동기 핸들러 경계 감각.

장르·리듬. 장르 ④ 따라만들기(경계 선거트 프로토콜 어휘)·순차(2B). 답지 12 / 문제지 11.

노트 읽는 순서 (signal-message-bus/notes/). posix-signal (정독) + 짝 reference-impl.

재구성·커밋 활용법. 000(프로토콜 어휘 헤더: MT_ZERO_SIGNAL / MT_ONE_SIGNAL / MT_ACK_SIGNAL, L2) → 001(server PID 출력·pause()) → **002~003(MSB-first 바이트 조합·byte→bit 송신, L3) → 004(NUL 종료자=메시지 경계, L3) → 005~006(sigsuspend 기반 ACK·ACK timeout, L3) → 007(경쟁 sender 거절·NACK, L3: 단일 활성 송신자) → 008~010(고정)**. 002~007이 L3 핵심.

L3 — 손으로 백지 재구성할 것. % 뒤 flags→width→precision→specifier 순서 소비, 변환 9종과 flag 조합의 우선순위(width→precision 상호작용, 부호+0패딩 동시), va_list 전달, counted-write sticky error.

검증. make → make test (snprintf 와 바이트·반환값 비교).

합정. length modifier-locale-stdout 외 출력·첫 커밋 체크리스트 밖 변환은 범위 밖.

완료 바 → 다음으로. width/precision/flag 우선순위를 백지에서 표로 재현하고 sticky error를 설명되면 다음.

5. thread-dining — C 결가지 ③: pthread·동시성(philosophers)

무엇/왜. dining philosophers. pthread-mutex-monitor 패턴으로 교착(deadlock)을 피하며 동시성을 푼다. 이 트랙에서 얻는 것. 락 소유권·교착 회피·상태 vs 출력 분리 — 동시성 L3의 첫 본격 훈련.

장르·리듬. 장르 ④ 따라만들기(골격 선거트 공통 타입)·순차(2B). 답지 19 / 문제지 17.

노트 읽는 순서 (thread-dining/notes/). posix-pthread → posix-time-io → awk (로그 검증).

재구성·커밋 활용법. 000(공통 타입 t_config / t_philo / t_table, L2: table이 fork/philo 소유) → 001~002(ms helper-deadline 풀링) → **003~004(state_mutex vs print_mutex 책임 분리, L3) → 005~008(철학자 루틴 출력 fork 순서·monitor·수명주기, L3) → 009~012(단일 철학자·meal limit 경계) → 013~017(고정)**. 003~008이 L3 — 덮고 직접 구현 후 답지와 diff.

L3 — 손으로 백지 재구성할 것. 출력 fork 획득 순서로 **deadlock 회피**, state_mutex (종료 플래그·식사 상태) vs print_mutex (로그) 분리, monitor는 main 스레드에서 직접 실행, deadline 기반 sleep 루프와 meal limit 즉시 종료.

검증. make test (실제 바이너리 실행, 종료 코드·로그 정규식 검증).

합정. bonus(semaphore/process)는 **실패**하는 target. 철학자 200명 초과는 거절. 로그 순서 결정성은 보장 안 됨(타이밍 의존).

완료 바 → 다음으로. 교착이 왜 안 나는지(출력 순서)와 두 mutex의 책임 경계를 백지로 방어되면 다음.

6. stack-sort — C 결가지 ④: 정렬 알고리즘(push_swap·Python)

무엇/왜. 42 push_swap. 11개 스택 명령만으로 정렬하는 C 바이너리 + 검증 C 바이너리 + Python 테스트 러너. 알고리즘 설계와 "독일 적용기로 출력 검증하는 법"을 함께 배운다. 이 트랙에서 얻는 것. 좌표 압축·radix 같은 알고리즘 사고 + 검증 자동화. (이 트랙의 유일한 Python 접점.)

장르·리듬. 장르 ③ 순제산(알고리즘 설계) + ④ 따라만들기·순차(2B). 답지 23 / 문제지 21.

노트 읽는 순서 (stack-sort/notes/). python (C 프로그램을 구동·검증하는 도구 측면).

재구성·커밋 활용법. 000~001(이중 바이너리 공용 소스 계약, L3: 공통 stack 연산 공유) → 002~004(스택 저장소·swap/push-rotation, L3: values와 ranks 나란히 보관) → **005~007(파서: token-signed int 검증·중복 거절·rank 압축, L3: 좌표 압축) → 008~012(tiny sort·LSD radix-main orchestration, L3: 이원화·emit 플래그) → 013~015(checker) → 016~020(자동 검증)**. 알고리즘 핵심(005~012)을 손으로 다시 설계한다.

L3 — 손으로 백지 재구성할 것. 좌표 압축(값 범위 무관 radix), LSD radix 전략, tiny sort 패턴 붓기, 두 바이너리가 같은 primitive 공유, emit 플래그로 출력 제어.

검증. make → make test (Python 러너: 파서/checker/정렬/stress).

합정. 명령 수 **최적화**가 목표가 아니라 **안정적 정렬 전략**. int 범위만. 시간 압박이면 결가지 중 이 한 편만 남기고 축소해도 된다(결가지는 서로 독립이라 떼어 써도 된다).

완료 바 → 다음으로. 좌표 압축 + radix를 백지로 재설계하고 checker와의 분리를 설명되면, 이제 C++로.

7. ★ pong-pong — 캡스톤: 풀스택 실시간 웹(취업 산출물)

무엇/왜. 실시간 Pong 매칭 MVP. TypeScript + Next.js(App Router) + Fastify(WebSocket) + PostgreSQL(Kysely) + Docker Compose 풀스택. 핵심은 서버 권위 경기 계산과 관계형 저장. CS 기본(특히 irc의 이벤트 루프·소켓)을 채용 시장이 보는 웹 스택으로 번역하는 자리다. 이 트랙에서 얻는 것. 풀스택 산출물 + TypeScript. 이후 프론트 트랙의 TS 선행이 여기서 채워진다. 선행은 M0뿐 — TS는 레포 자체의 자족 온램프(docs/notes/systems/node-typescript.md)로 들어간다.

장르·리듬. 장르 ①(코드 재구성, shared 계약 선행)·순차(2B). 답지 86 / 문제지 68 — **대형 레포다**. 전부 같은 무게로 들지 마라. L3만 길게, 나머지는 묶여서 빠르게.

노트 읽는 순서 (pong-pong/notes/). nodejs → tsx · fastify-cors → fastify-cookie → fastify-websocket → ws · postgresql → pg → kysely · caddy · smoke-scripts → playwright. pong이 새로 가르치는 핵심 = PostgreSQL·WebSocket — 이 두 묶음을 길게.

재구성·커밋 활용법. 묶음으로 본다 — 000~005(shared HTTP/game/ws 계약 선행, L3: 계약이 프론트/백 공유) → 006~014(PostgreSQL schema·AppRepository·web/api scaffold) → **015~031(Canvas-based WebSocket GameHub·lobby/play, L3: 서버 권위 경기 상태) → 032~041(Docker/Caddy·smoke-Playwright) → 042~063(action API·chat-metrics-tournament-admin-presence, L3: 서버 권위 경기 상태) → 064~084(truth fix·세션/토론 안정화)**. L3는 "실용 분리"와 "shared 계약·auth 경계" — 이 커밋들만 손으로 재구성하고, CRUD·화된 배션류는 답지 펼쳐 읽고 통과.

L3 — 손으로 백지 재구성할 것. 서버 권위 경기 상태(점수·위치·승패를 서버가 계산, 클라 입력 불신), GameHub 의 tick loop + 이벤트 기반 broadcast, shared 계약이 프론트/백을 함께 묶는 구조, AppRepository 로 저장소 추상화 + 트랜잭션, auth 경계(익명 세션 즉시 null, 401 토큰 즉시 제거).

검증. npm run typecheck/build/test + make test (smoke + Playwright E2E) + Docker Compose 실행. 화면↔코드를 왕복하며 실제로 띄워 본다 (npm dev).

합정. 외부 OAuth는 환경변수 분리만(저장소 미포함, dev login 기본). 외부 사용자 트래픽용 아님. web unit test는 미완성. **85커밋 전부 재구성은 과투자** — 서버 권위 계약·auth만 길게.

완료 바 → 트랙 완주. Docker Compose로 풀스택이 뜨고, 서버 권위 모델·shared 계약·auth 경계를 백지로 방어되면 42 트랙 완주. GitHub에 push해 커리어 산출물로 남긴다.

커밋 메모. npm 모노레포라 변경 범위가 넓다 — dual-form은 docs/commits · docs/practice 만 끝날 add, 구현 커밋과 메타-문서 커밋의 phase 경계를 섞지 않는다(상단 <커밋 규율>).

다음 — 분기, 그리고 병렬성

pong을 닫으면 **백엔드** 트랙 또는 **프론트** 트랙으로 간다. 둘 다 M0(linux-admin)-TS(pong)를 이미 끝낸 상태에서 시작하므로, 그 두 트랙은 이 선행을 다시 다루지 않는다.

- **트랙 간 병렬**: 42 스파인 / 백 / 프론트는 서로 **하드 의존이 없다**(공통 게이트는 M0뿐). 그래서 C5(42)와 웹(백·프론트)은 *비대칭*이라 원하면 병렬도 가능하다. 다만 이 트랙 계획은 **42를 먼저 완주한 뒤** 분기하는 단선이다.

- **분기 후 트랙 내부의 병렬성**은 각 문서 말미에서 다룬다(백: 신뢰성→캡스톤 순차·필수, 프론트: 캡스톤이 자족이라 신뢰성과 병행 가능).